

Zero Knowledge (ZK) solutions

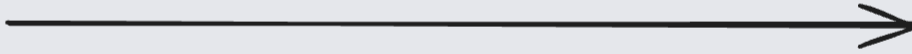
Prepared by Kirill Sizov

Prover and verifier



Prover

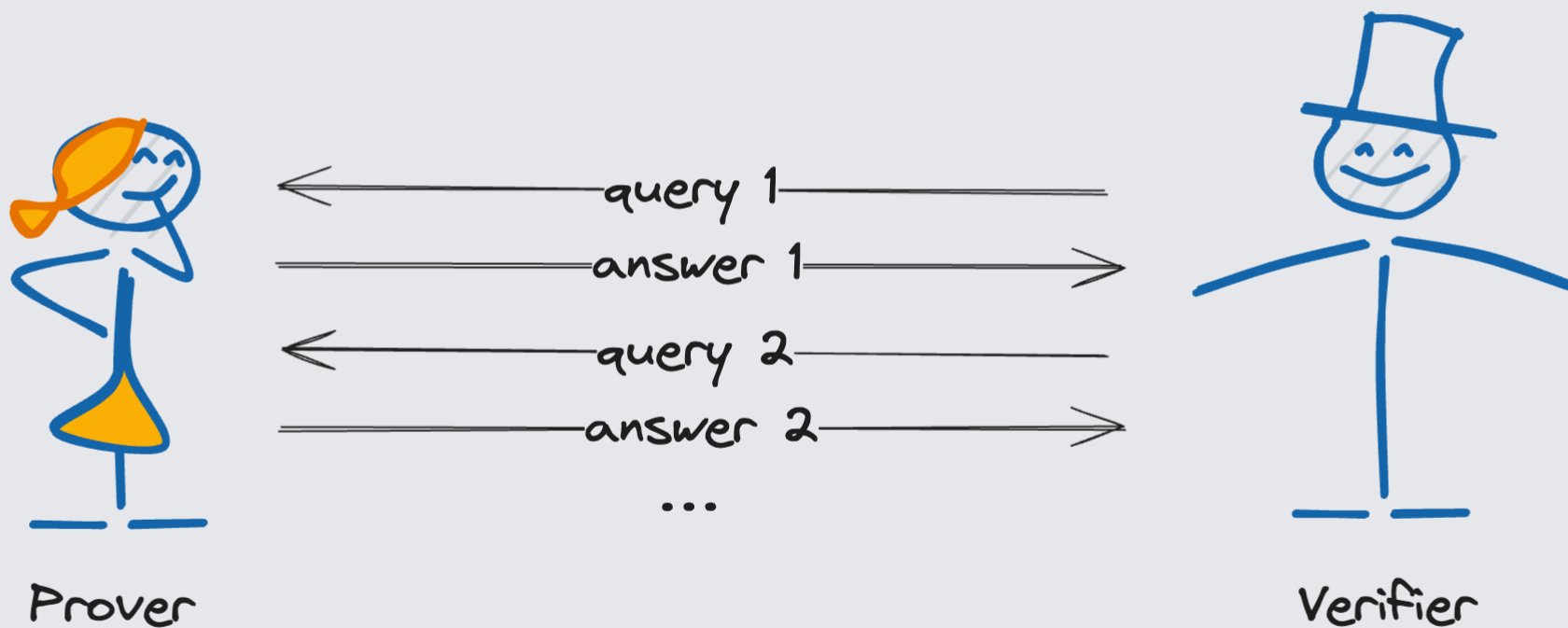
sends proof of the claim



Verifier

accepts / rejects
the proof

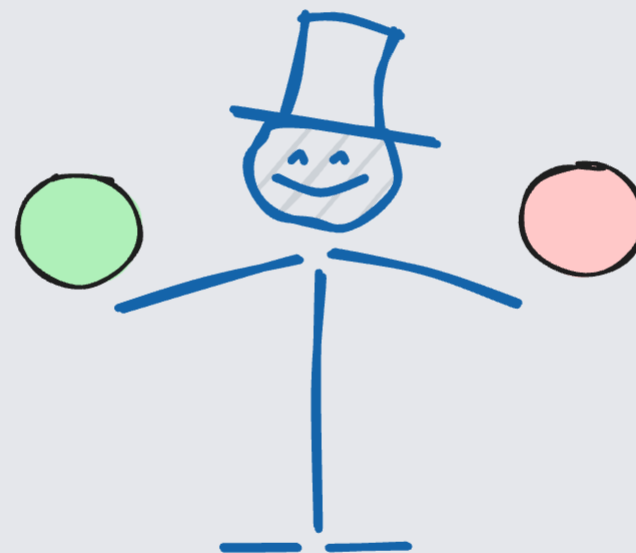
Interactive proof model



Example: proving two balls are different colors to colorblind verifier

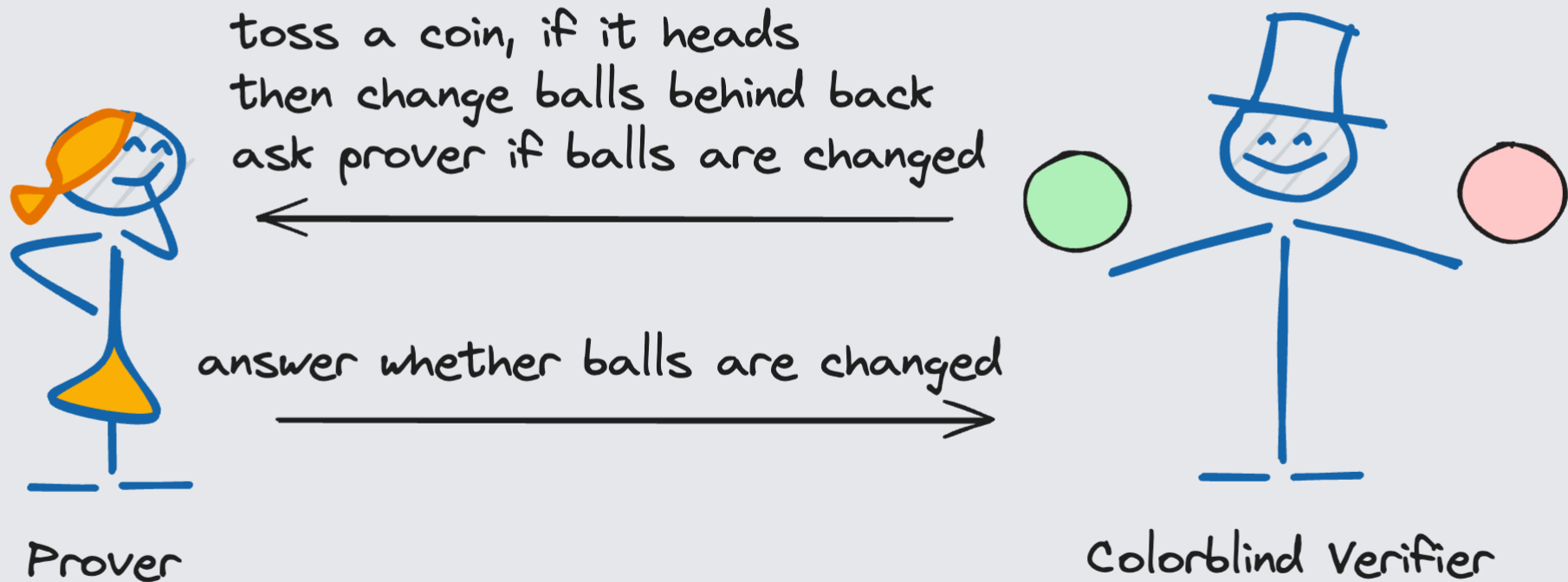


Prover



Colorblind Verifier

Example: proving two balls are different colors to colorblind verifier



Example: proving quadratic residue

- $QR = \{(N, y) : \exists x : y \equiv x^2 \pmod{N}\}$
- Trivial proof: provide x
- Can we proof without revealing x ?

Example: proving quadratic residue

Fiat–Shamir protocol

$$QR = \{(N, y) : \exists x : y \equiv x^2 \pmod{N}\}$$

Prover

- Chooses random $1 \leq r \leq N : (r, N) = 1$
- Sends $s = r^2 \pmod{n}$ to verifier

Verifier

- Flips a coin and sends the result to prover

Prover

- if it heads return $\sqrt{s} = r \pmod{N}$
- if it tails return $\sqrt{sy} = r\sqrt{y} \pmod{N}$

Interactive proof: properties

- Completeness – valid proofs are always accepted by the verifier.
- Soundness – invalid proofs are almost always rejected.

What is zero-knowledge

What the verifier can compute **after** interaction

= What the verifier could compute **before** interaction

zkSNARK

zero-**k**nowledge **S**uccinct **N**on-interactive **A**Rguments of **K**nowledge

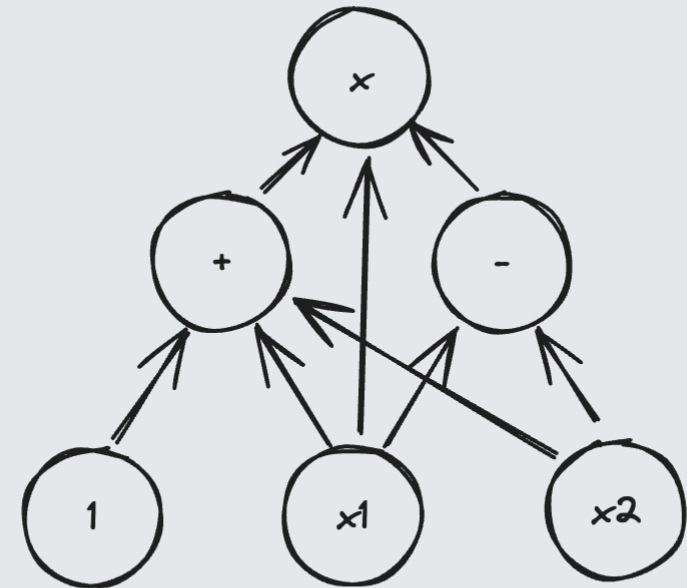
- Succinct: The proofs are small and easy to verify, regardless of the complexity of the statement.
- Non-Interactive: Does not require multiple rounds of communication between prover and verifier, making it efficient for decentralized systems.
- Argument of Knowledge: Proof that the prover knows the inputs from which the output was derived.

Circuits

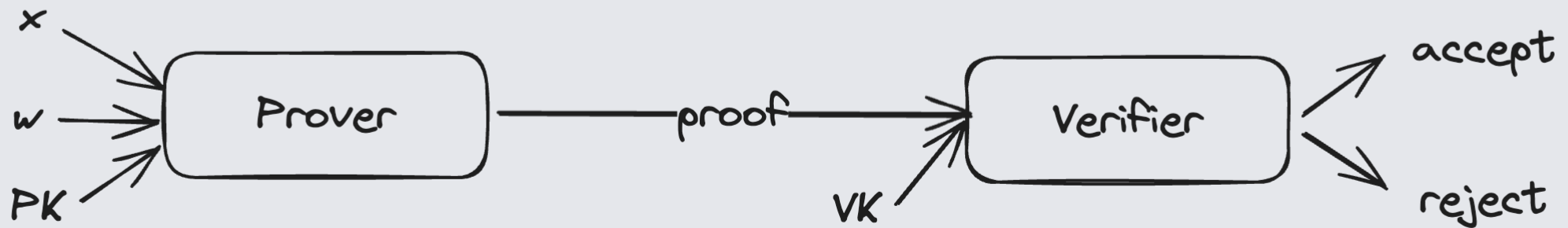
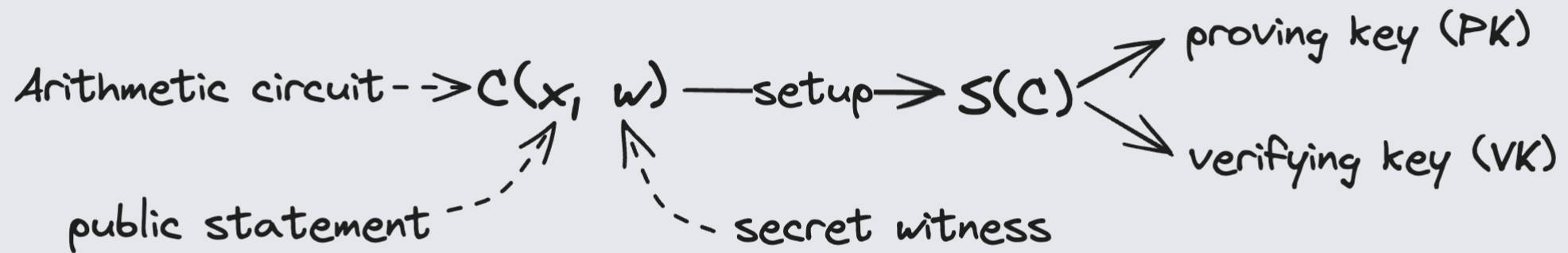
Arithmetic circuit

- Directed Acyclic Graph (DAG) with internal nodes representing operations (+, -, *) and leaves serving as input values.

$$(1 + x_1 + x_2) * x_1 * (x_1 - x_2)$$



SNARK



Setup types

- Trusted setup per circuit
 - Relies on a secret that should be kept from provers.
 - Examples: Groth16, Pinocchio, GKR.
- Universal Trusted Setup:
 - Can be reused across multiple circuits or applications.
 - Examples: PLONK, Sonic, Marlin.
- Transparent Setup
 - Don't use secret data.
 - Examples: zk-STARKs, Halo, Bulletproofs.

Groth16

Basic zero knowledge proof with elliptic curves

- Claim: I know two values x and y such that $x + y = C$
- Prover: Send $A = xG$ and $B = yG$ to verifier, where G is generator
- Verifier: Checks that $A + B = C \cdot G$

Bilinear pairing

- Bilinear pairing allows to prove that a 3rd number is the product of the original numbers without revealing them.
- Bilinear pairing is a map $e : G_1 \times G_2 \rightarrow G_T$ which satisfy bilinearity, non-degeneracy, efficiency.

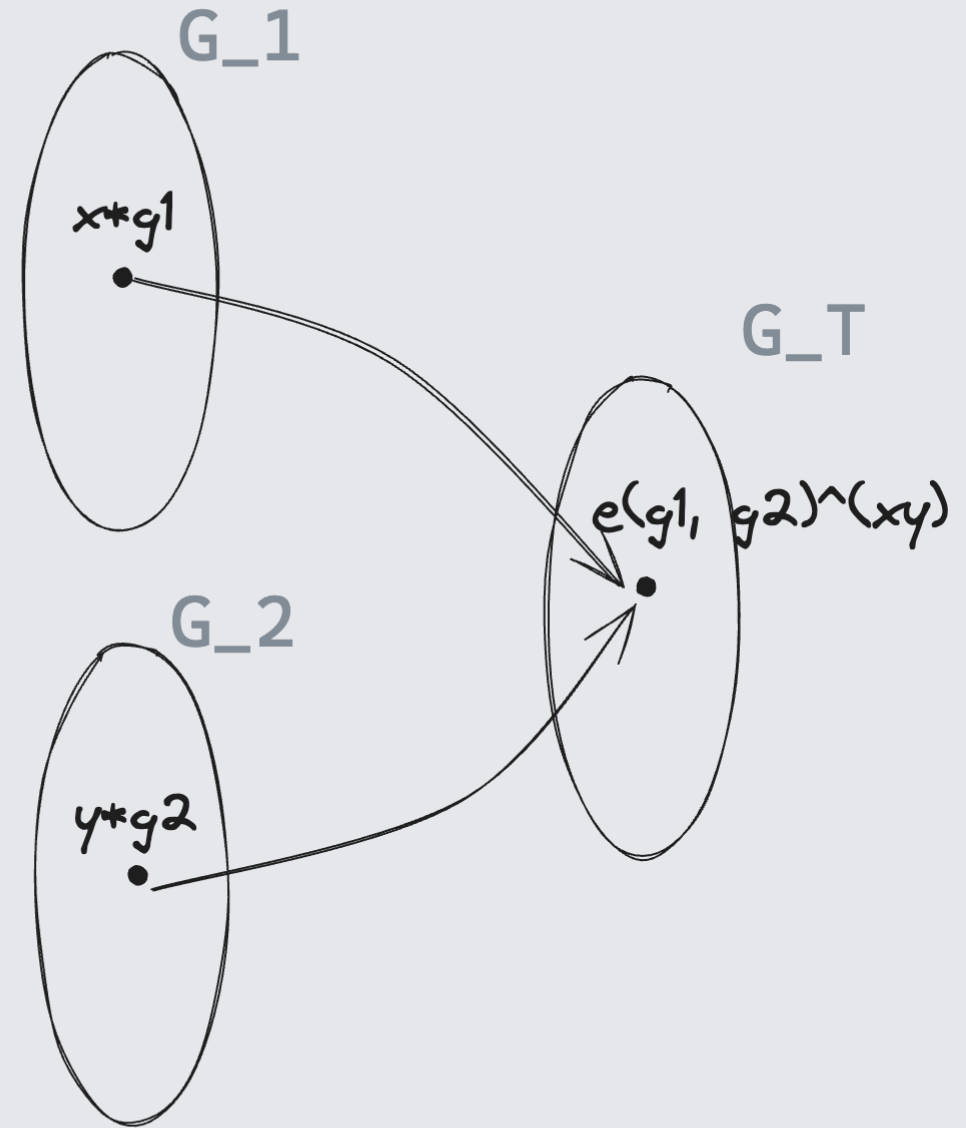
Bilinear pairing

- Bilinearity: $e(P + Q, S) = e(P, S) \cdot e(Q, S)$ and $e(P, S + T) = e(P, S) \cdot e(P, T)$
- Non-degeneracy: $e(P, Q) = 1_{G_T}$ only when $P = 0_{G_1}$ or $Q = 0_{G_2}$

From bilinearity, it follows that if g_1 is a generator of G_1 , and g_2 is a generator of G_2 , then $e(Ag_1, Bg_2) = e(g_1, g_2)^{AB}$.

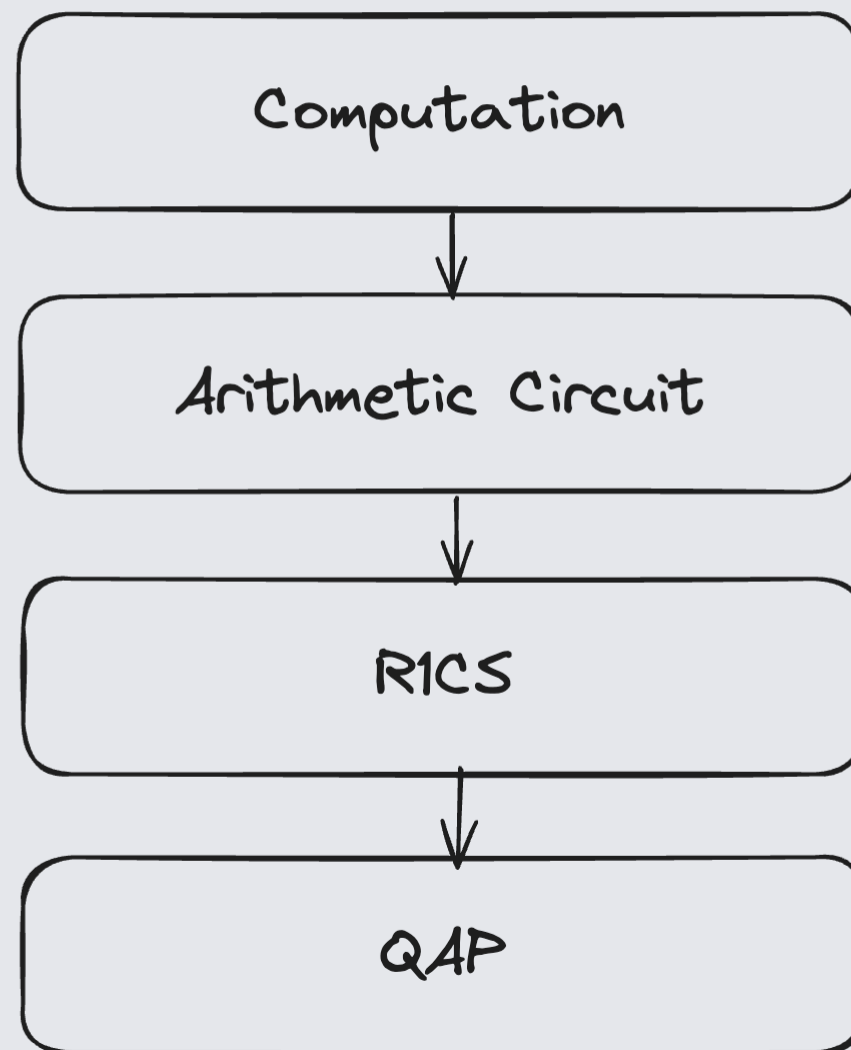
Bilinear pairing

- Claim: I know two values x and y such that $xy = D$
- Prover: Send $A = xg_1$ and $B = yg_2$ to verifier
- Verifier: Checks that $e(A, B) = e(g_1, g_2)^D$



* g_1 and g_2 are generators in G_1 and G_2

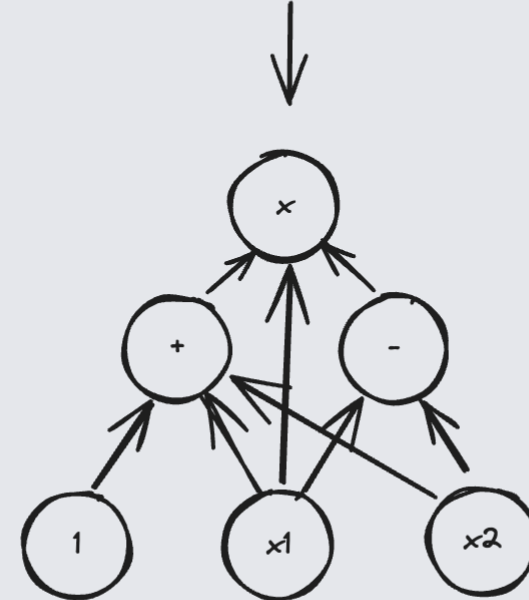
General steps



Flattening

In this step we convert the original code, which may contain arbitrarily complex statements and expressions, into arithmetic circuit.

```
pragma circom 2.1.6;  
  
template Example() {  
  signal input x1;  
  signal input x2;  
  signal output out;  
  
  out <== (1 + x1 + x2) * x1 * (x2 - x1);  
}  
  
component main = Example();
```



Rank one constrain system (R1CS)

- The witness vector is $1 \times n$ vector that contains the value of all the input variables, the output variable, and the intermediate values
- $out = x^2y$
 - $v_1 = x \cdot x$
 - $out = v_1 \cdot y$
 - witness vector: $[1, out, x, y, v_1]$

Rank one constrain system (R1CS)

- An R1CS is a sequence of groups of three vectors (a, b, c) (each logic gate is converted into one group of vectors).
- The solution to an R1CS is a vector w , where w must satisfy the equation $(a \cdot s) \circ (b \cdot s) = c \cdot w$
- We can combine all equations into one equation $(A \cdot w) \circ (B \cdot w) = C \cdot w$
- For example $out = x^2y$
 - $w = [1, out, x, y, v_1]$
 - $A = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$
 - $B = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$
 - $C = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$

ZK R1CS

$$\begin{bmatrix} l_{1,1} & \dots & l_{1,m} \\ \vdots & \ddots & \vdots \\ l_{n,1} & \dots & l_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix} \odot \begin{bmatrix} r_{1,1} & \dots & r_{1,m} \\ \vdots & \ddots & \vdots \\ r_{n,1} & \dots & r_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix} = \begin{bmatrix} o_{1,1} & \dots & o_{1,m} \\ \vdots & \ddots & \vdots \\ o_{n,1} & \dots & o_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix}$$



$$\begin{bmatrix} l_{1,1} & \dots & l_{1,m} \\ \vdots & \ddots & \vdots \\ l_{n,1} & \dots & l_{n,m} \end{bmatrix} \begin{bmatrix} [s_1 G_1]_1 \\ \vdots \\ [s_m G_1]_1 \end{bmatrix} = \begin{bmatrix} l_{1,1}[s_1 G_1] & + \dots & + l_{1,m}[s_m G_1] \\ \vdots & \ddots & \vdots \\ l_{n,1}[s_1 G_1] & + \dots & + l_{n,m}[s_m G_1] \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m l_{1,i}[s_i G_1]_1 \\ \sum_{i=1}^m l_{2,i}[s_i G_1]_1 \\ \vdots \\ \sum_{i=1}^m l_{n,i}[s_i G_1]_1 \end{bmatrix}$$

$$\begin{bmatrix} r_{1,1} & \dots & r_{1,m} \\ \vdots & \ddots & \vdots \\ r_{n,1} & \dots & r_{n,m} \end{bmatrix} \begin{bmatrix} [s_1 G_2]_2 \\ \vdots \\ [s_m G_2]_2 \end{bmatrix} = \begin{bmatrix} r_{1,1}[s_1 G_2]_2 & + \dots & + r_{1,m}[s_m G_2]_2 \\ \vdots & \ddots & \vdots \\ r_{n,1}[s_1 G_2]_2 & + \dots & + r_{n,m}[s_m G_2]_2 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m r_{1,i}[s_i G_2]_2 \\ \sum_{i=1}^m r_{2,i}[s_i G_2]_2 \\ \vdots \\ \sum_{i=1}^m r_{n,i}[s_i G_2]_2 \end{bmatrix}$$

ZK R1CS

$$\begin{bmatrix} l_{1,1} & \dots & l_{1,m} \\ \vdots & \ddots & \vdots \\ l_{n,1} & \dots & l_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix} \odot \begin{bmatrix} r_{1,1} & \dots & r_{1,m} \\ \vdots & \ddots & \vdots \\ r_{n,1} & \dots & r_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix} = \begin{bmatrix} o_{1,1} & \dots & o_{1,m} \\ \vdots & \ddots & \vdots \\ o_{n,1} & \dots & o_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix}$$



$$\begin{aligned} & \text{pairing} \left(\begin{bmatrix} \sum_{i=1}^m l_{i,1} [s_i G_1]_1 \\ \sum_{i=1}^m l_{i,1} [s_i G_1]_1 \\ \vdots \\ \sum_{i=1}^m l_{i,1} [s_i G_1]_1 \end{bmatrix}, \begin{bmatrix} \sum_{i=1}^m r_{i,1} [s_i G_2]_2 \\ \sum_{i=1}^m r_{i,1} [s_i G_2]_2 \\ \vdots \\ \sum_{i=1}^m r_{i,1} [s_i G_2]_2 \end{bmatrix} \right) \stackrel{?}{=} \text{pairing} \left(\begin{bmatrix} \sum_{i=1}^m o_{i,1} [s_i G_1]_1 \\ \sum_{i=1}^m o_{i,1} [s_i G_1]_1 \\ \vdots \\ \sum_{i=1}^m o_{i,1} [s_i G_1]_1 \end{bmatrix}, \begin{bmatrix} G_2 \\ G_2 \\ \vdots \\ G_2 \end{bmatrix} \right) \\ & = \begin{bmatrix} [\text{pairing}(\sum_{i=1}^m l_{i,1} [s_i G_1]_1, \sum_{i=1}^m r_{i,1} [s_i G_2]_2)]_{12} \\ [\text{pairing}(\sum_{i=1}^m l_{i,2} [s_i G_1]_1, \sum_{i=1}^m r_{i,2} [s_i G_2]_2)]_{12} \\ \vdots \\ [\text{pairing}(\sum_{i=1}^m l_{i,n} [s_i G_1]_1, \sum_{i=1}^m r_{i,n} [s_i G_2]_2)]_{12} \end{bmatrix} \stackrel{?}{=} \begin{bmatrix} [\text{pairing}(\sum_{i=1}^m o_{i,1} [s_i G_1]_1, G_1)]_{12} \\ [\text{pairing}(\sum_{i=1}^m o_{i,2} [s_i G_1]_1, G_2)]_{12} \\ \vdots \\ [\text{pairing}(\sum_{i=1}^m o_{i,n} [s_i G_1]_1, G_2)]_{12} \end{bmatrix} \end{aligned}$$

Quadratic Arithmetic Programs (QAP)

- QAP is a system of equations where the coefficients are polynomials and a valid solution results in a single polynomial equality
- QAPs make ZK-Snarks succinct
- R1CS evaluation is not succinct due to the many operations of matrix multiplication
- The key idea behind QAP
 - Operations in R1CS (addition and Hadamard product) form an algebraic ring when viewed as a set of column vectors
 - Polynomials under addition and multiplication are rings
 - There exists an easily computable homomorphism from R1CS to polynomials

QAP polynomial construction

- R1CS equation: $(A \cdot w) \circ (B \cdot w) = C \cdot w$
- Build a polynomial for each column in A , B , C and one polynomial for w
- For each vector v , construct a polynomial passing through points (i, v_i)
- Apply Lagrange interpolation to build such polynomials.

QAP polynomial construction

- $A \rightarrow A(x) = [a_1(x), a_2(x), \dots, a_m(x)]$
- $B \rightarrow B(x) = [b_1(x), b_2(x), \dots, b_m(x)]$
- $C \rightarrow C(x) = [c_1(x), c_2(x), \dots, c_m(x)]$
- $(A \cdot w) \circ (B \cdot w) = C \cdot w \Rightarrow (A(x) \cdot w) \cdot (B(x) \cdot w) = C(x) \cdot w \ \forall x = 1, \dots, n$

QAP checks

- $(A(x) \cdot w) \cdot (B(x) \cdot w) = C(x) \cdot w \ \forall x = 1, \dots, n$
- $(A(x) \cdot w) \cdot (B(x) \cdot w) - (C(x) \cdot w) = 0 \ \forall x = 1, \dots, n$
- $(A(x) \cdot w) \cdot (B(x) \cdot w) - (C(x) \cdot w) \dot{=} T(x) = (x - 1) \dots (x - n)$
- $\exists H(x) : (A(x) \cdot w) \cdot (B(x) \cdot w) - (C(x) \cdot w) = T(x) \cdot H(x)$

Prover can calculate $H(x) = \frac{(A(x) \cdot w) \cdot (B(x) \cdot w) - (C(x) \cdot w)}{T(x)}$

Schwartz–Zippel lemma

The Schwartz–Zippel lemma ensures that in QAPs, if a polynomial identity $A(x) \cdot B(x) \neq C(x) + H(x)T(x)$, it will likely be detected by evaluating at a random point r .

Adding homomorphic hiding

- Trusted setup: choose some x and calculate $xG, x^2G, \dots, x^nG$
 - prover shouldn't know the x !

$$A' = E(A(x) \cdot w) = a_d E(x^d) + a_{d-1} E(x^{d-1}) + \dots + a_1 E(x) + a_0 = a_d (x^d G) + a_{d-1} (x^{d-1} G) + \dots + a_1 (xG) + a_0 G$$

$$B' = E(B(x) \cdot w) = b_d E(x^d) + b_{d-1} E(x^{d-1}) + \dots + a_1 E(x) + b_0 = b_d (x^d G) + b_{d-1} (x^{d-1} G) + \dots + b_1 (xG) + b_0 G$$

$$C' = E(C(x) \cdot w + H(x) \cdot T(x)) = c_d E(x^d) + c_{d-1} E(x^{d-1}) + \dots + c_1 E(x) + c_0 = c_d (x^d G) + c_{d-1} (x^{d-1} G) + \dots + c_1 (xG) + c_0 G$$

- Need to check only one pairing `pairing(A', B') = C'`

Blockchain applications

- Privacy preserving transactions
 - anonymous transfers (e.g. Tornado Cash) or private smart contracts (e.g. Aztec)
- Scalability solutions
 - ZK-Rollups (e.g., StarkNet, zkSync, Polygon zkEVM)
- DeFi applications